# Solana Risk Monitor - System Architecture

## Table of Contents


1. Program Structure Visualization
2. Account Structure Mapping
3. External Dependencies and Integrations
4. Detailed Charting Techniques
5. Program Interaction Flow
6. Detailed Architecture Sections
7. Error Handling and Edge Cases
8. Evaluation Architecture


## System Overview

Real-time portfolio risk monitoring system for Solana DeFi traders with automated alerts based on personalized thresholds.

**Core Programs (4 Total)**

**1. User Configuration Program**

**Purpose**: Store user preferences and risk thresholds
**Key Instructions**:

- initialize_user() - Create user account
- update_thresholds(low, med, high) - Update risk settings
- read_config() - Fetch user preferences

**Account Created**: User Config PDA (53 bytes)

**2. Portfolio Analyzer Program**

**Purpose**: Fetch and analyze SPL token holdings
**Key Instructions**:

- fetch_balances() - Query all SPL token accounts
- calculate_concentration() - Compute Herfindahl Index
- compute_exposure() - Calculate USD value exposure

**Account Created**: Portfolio Snapshot PDA (~500 bytes)

**3. Risk Calculation Engine**

**Purpose**: Calculate composite risk scores
**Key Instructions**:

- analyze_volatility() - 30-day historical volatility
- compute_correlation() - Token pair correlations
- classify_risk() - LOW/MEDIUM/HIGH classification
- store_snapshot() - Save risk history

**Accounts Created**:

- Risk Score PDA (54 bytes)
- Risk History PDA (variable)

**4. Alert Management Program**

**Purpose**: Monitor thresholds and send alerts
**Key Instructions**:

- check_threshold(score) - Compare against user settings
- create_alert() - Generate alert event
- log_alert() - Store in history
- emit_event() - Publish to event bus

**External Dependencies**

**On-Chain**

- **Solana RPC**: Account queries, transaction submission
- **SPL Token Program**: Token balance reads
- **Pyth Price Oracle**: Real-time token prices

**Off-Chain**

- **PostgreSQL**: Historical data storage (90-day retention)
- **Redis Cache**: Token price caching
- **ML Service (FastAPI)**: Risk score prediction
- **Event Bus (WebSocket)**: Real-time notifications
- **Notification Service**: Discord/Email/Push delivery

## Technical Stack Summary

**On-Chain**: Anchor Framework, Rust, Solana Program Library

**Off-Chain**: TypeScript/Node.js, Python (ML), PostgreSQL, Redis

**Frontend**: React, Solana Wallet Adapter, WebSocket client

**Notifications**: Discord API, SendGrid, OneSignal

**Infrastructure**: Docker, cron, FastAPI

# Key Workflows

## Initial Setup

1. User connects wallet
2. Initialize user account (transaction)
3. Set risk thresholds: Low (0-33), Medium (34-66), High (67-100)
4. System begins monitoring every 5 minutes

## Risk Calculation Cycle (Every 5 Minutes)

1. Fetch all SPL token balances from user wallet
2. Fetch current prices from Pyth Oracle
3. Calculate portfolio value and concentration
4. Fetch 30-day historical volatility
5. ML service predicts risk components
6. Compute composite risk score: 0.4×Concentration + 0.4×Volatility + 0.2×Correlation
7. Classify risk level (LOW/MEDIUM/HIGH)
8. Store snapshot and risk score on chain

## Alert Trigger

1. Compare risk score against user thresholds
2. If score > threshold AND no active alert:
    a. Create alert event
    b. Log to alert history PDA
    c. Emit to event bus
    d. Send Discord webhook
    e. Send email notification
    f. Update UI via WebSocket
3. Implement 30-minute cooldown to prevent spam

# Cross-Program Invocation (CPI) Flow

```
User Config ← Portfolio Analyzer (read config)
        ↓
    Risk Engine (calculate risk)
        ↓
    Alert Manager (check threshold)
        ↓
    User Config (read thresholds)
```

## Timing and Scheduling

- **Risk Calculation**: Every 5 minutes (cron job)
- **Price Updates**: Pyth Oracle updates every ~400ms
- **Alert Cooldown**: 30 minutes between same-type alerts
- **Data Retention**: 90 days of historical snapshots
- **WebSocket Heartbeat**: Every 30 seconds

## Error Handling Strategy

**RPC Failures**

- Retry up to 3 times with 2-second delays
- Fallback to secondary RPC endpoints
- Log failure if all retries exhausted

**Oracle Data Stale (>60 seconds old)**

- Fallback to Jupiter aggregator API
- Continue with best available data

**ML Service Unavailable**

- Use statistical heuristics instead
- Calculate simple concentration + historical volatility
- System remains operational

**Transaction Failures**

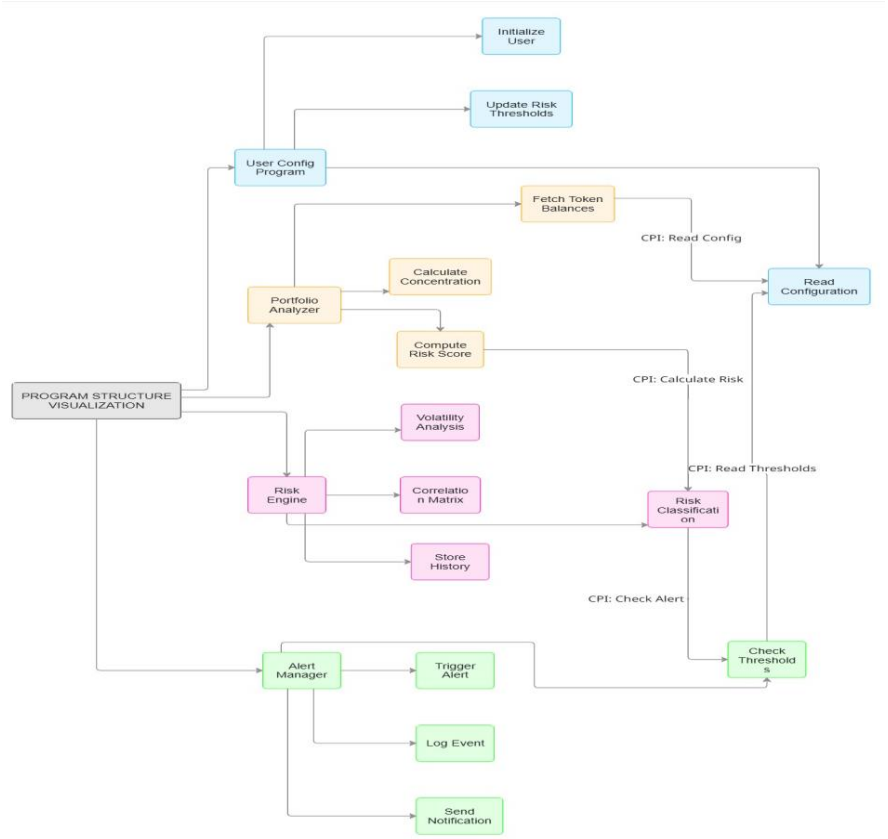
- Retry up to 2 times with 5-second delays
- Dynamically increase priority fees if congested
- Display clear error messages to user

**Account Size Limits**

- Implement circular buffer for alert history
- Overwrite oldest entries when limit reached
- Maintain max 50 alerts per user

## 1) Program Structure Visualization :

| Program | Primary Responsibility | Key Instructions | Dependencies |
|---|---|---|---|
| User Configuration Program | Store and manage user-defined risk preferences | initialize_user, update_thresholds, set_portfolio_params | None (Base Program) |
| Portfolio Analyzer Program | Fetch and analyze SPL token holdings | fetch_balances, calculate_concentration, compute_exposure | SPL Token Program, User Config |
| Risk Calculation Engine | Process risk metrics and classify risk levels | analyze_volatility, compute_correlation, classify_risk, store_snapshot | Historical Data Service, ML Service |
| Alert Management Program | Monitor thresholds and dispatch alerts | check_threshold, create_alert, emit_event, log_alert | Event Bus, Notification Service |






**Program Structure Visualisation Model**

## 2) Account Structure Mapping:

**The PDA Derivation Diagram Shows:**

**How the system creates unique addresses for each user's data**

Starting with YOUR wallet address (User Pubkey), the system generates 4 different PDA addresses:

1. **User Config PDA** ← Created from: ["user-config" + your_wallet_address]
2. **Portfolio Snapshot PDA** ← Created from: ["portfolio-snapshot" + your_wallet_address + timestamp]
3. **Risk Score PDA** ← Created from: ["risk-score" + your_wallet_address]
4. **Alert History PDA** ← Created from: ["alert-history" + your_wallet_address]

**Why this matters**:

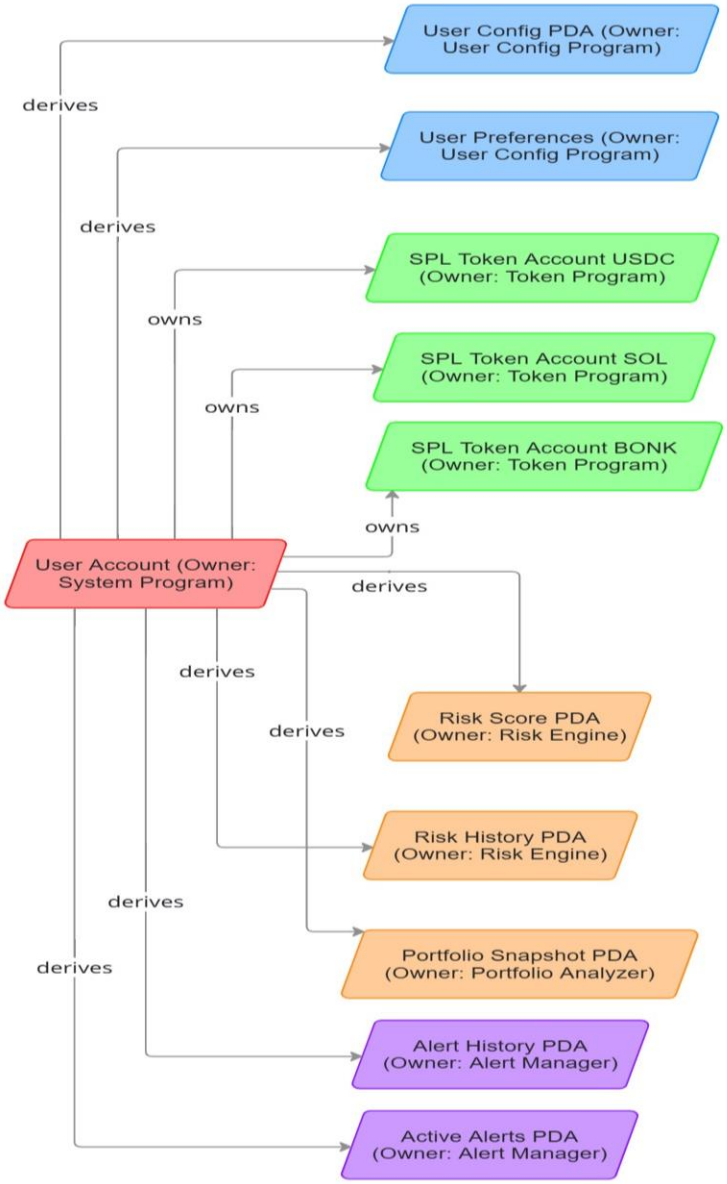
- Your data is stored at predictable addresses
- No one else can access or modify your specific accounts
- The program can always find your data using just your wallet address

**Simple Analogy:**

Think of it like a filing system:

- **Your wallet** = Your employee ID badge
- **User Config PDA** = Your personnel file (preferences, settings)
- **Portfolio Snapshot PDA** = Daily time sheets (what you did each day)
- **Risk Score PDA** = Your current performance review (latest status)
- **Alert History PDA** = Log of all warnings you've received

## Account Hierarchy and Data Model






## 3) External Dependencies and Integrations :

### 1. Solana RPC Integration

**Purpose**: Fetch on-chain account data and token balances

**How it works**:

- Makes HTTP/WebSocket requests to Solana nodes
- Key endpoints:
  - getAccountInfo - Get specific account details
  - getTokenAccountsByOwner - List all SPL tokens in wallet
  - getProgramAccounts - Query program-owned accounts
- Rate limit: 100 requests/second
- **Fallback**: Auto-switches to QuickNode or Helius if primary RPC fails

**Analogy**: Like an ATM checking your bank balance. If one's down, try another branch.

### 2. Pyth Price Oracle

**Purpose**: Real-time token price feeds

**How it works**:

- Reads price data directly from Pyth's on-chain accounts
- Updates every **400ms** (2.5x per second)
- Supported tokens: SOL, USDC, USDT, BONK, RAY, ORCA, JUP
- **Fallback**: If data is stale (>60 seconds), uses Jupiter aggregator API

**Analogy**: Stock ticker that updates constantly. If ticker breaks, check backup price board.

### 3. Historical Data Service (PostgreSQL)

**Purpose**: Store portfolio snapshots and risk history for trend analysis

**Database schema**:

- portfolio_snapshots - What you owned at each timestamp
- risk_scores - Historical risk calculations with component breakdown
- alert_events - Complete alert log with details

**Data retention**: 90-day rolling window (auto-deletes old data)

**Indexing**: Organized by user_id + timestamp for fast queries

**Example**: "On Feb 5 at 2pm, you had 100 SOL + 1000 BONK = $8,500, risk score: 65 (MEDIUM)"

### 4. ML Inference Service

**Purpose**: AI-powered risk prediction and classification

**Tech stack**: Python FastAPI service in Docker container

**Three models**:

1. **LSTM (Volatility Prediction)** - Analyzes price movement patterns over time
2. **Statistical Analysis (Correlation)** - Detects which tokens move together
3. **Random Forest (Classification)** - Categorizes risk as LOW/MEDIUM/HIGH

**Inputs**: Token holdings, historical prices, market data

**Outputs**: Risk scores (0-100 scale)

**Frequency**: Recalculates every 5 minutes

**Example**: "Your 80% BONK concentration + high volatility = 75 risk score (HIGH)"

**5. Event Bus & Notifications**

**Purpose**: Real-time alert delivery system

**Event Bus (WebSocket)**

Broadcasts on 3 channels:

- risk_updates - Score changes
- alert_triggers - Threshold breaches
- portfolio_changes - Balance updates

**Notification Channels**

**Discord Webhooks** (Primary)

- Instant alerts to server/DM
- Example: "HIGH RISK: Score 78 (threshold: 70)"

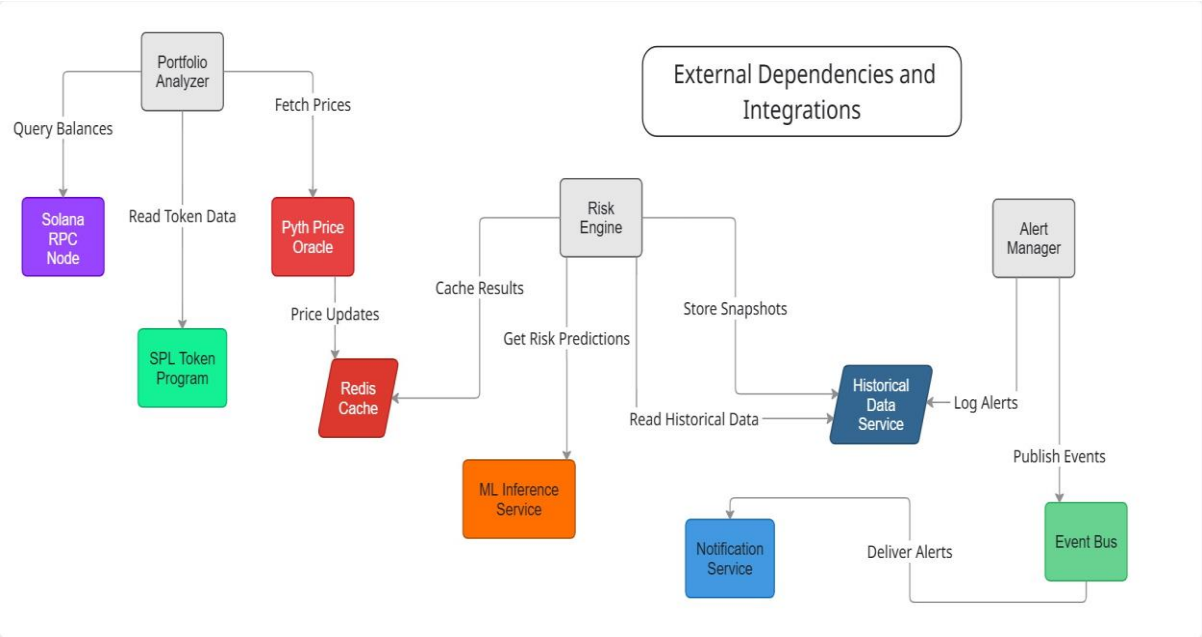
**Email (SendGrid)**

- Detailed formatted alerts
- Example: "Portfolio concentration increased to 85%"

**Push (OneSignal)**

- Mobile notifications
- Example: " Portfolio Risk: HIGH"

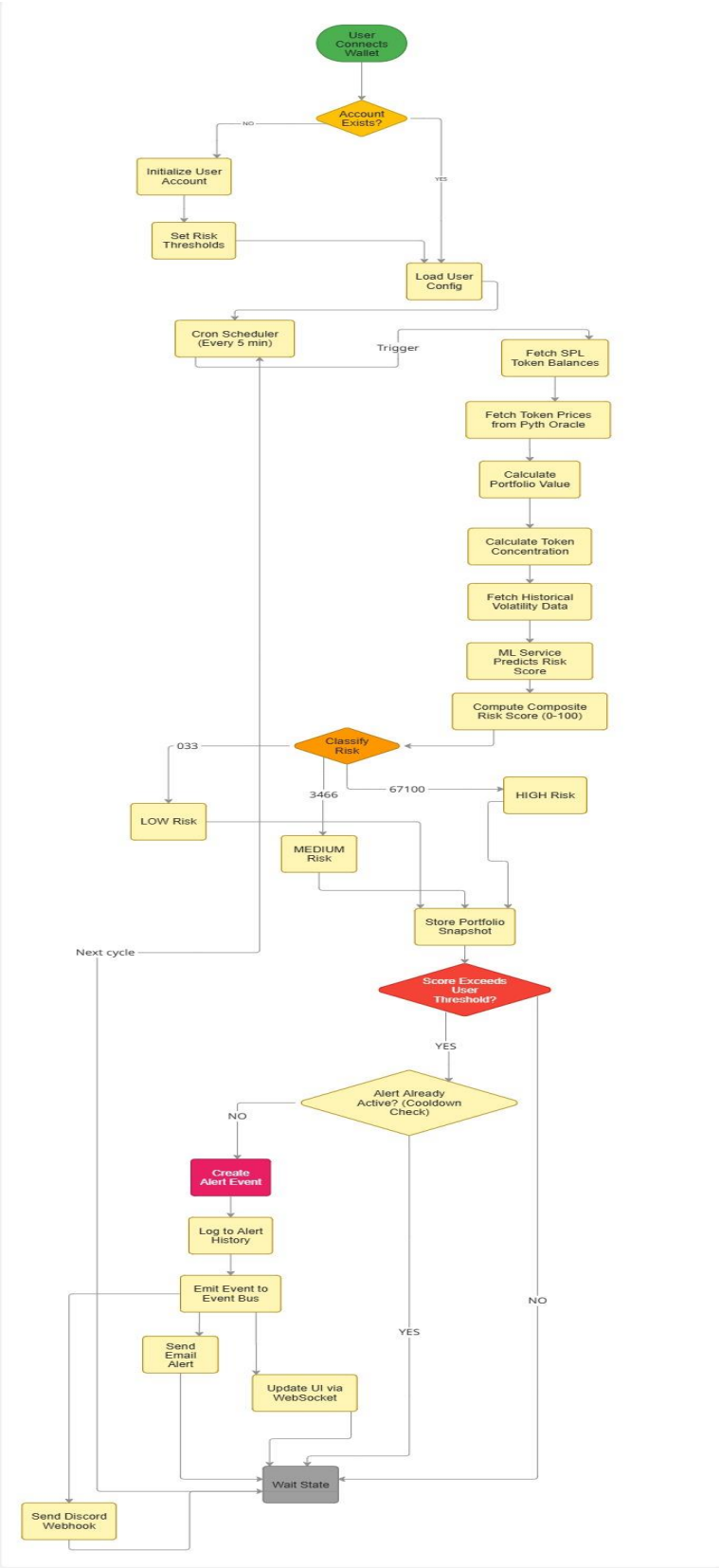
**FLOW > Wallet → RPC → Pyth → ML → PostgreSQL → Event Bus → You**



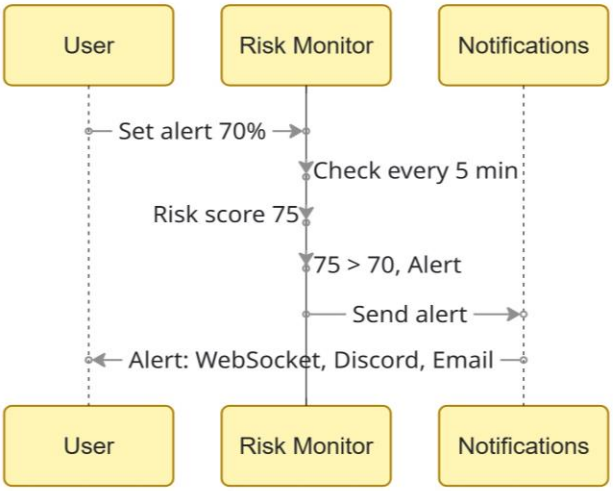

**External Dependencies and Integrations Model**

## 4) Detailed Charting Techniques :

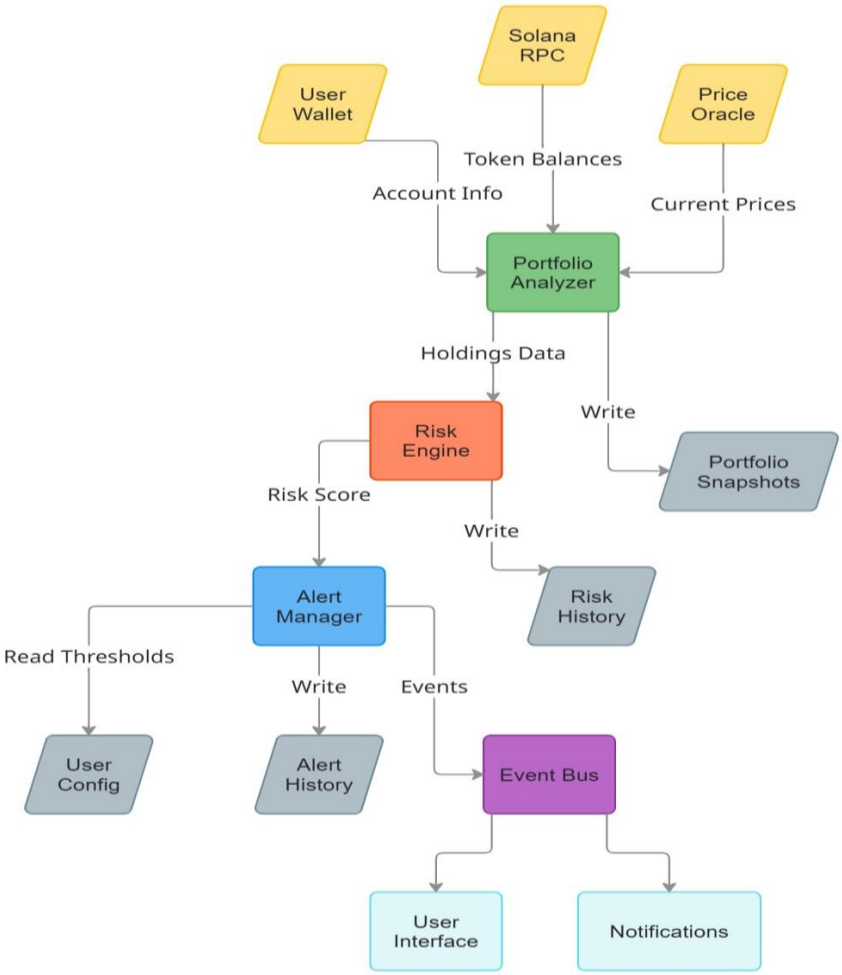
### System-Wide Flowchart :




## 5) Program Interaction Flow:

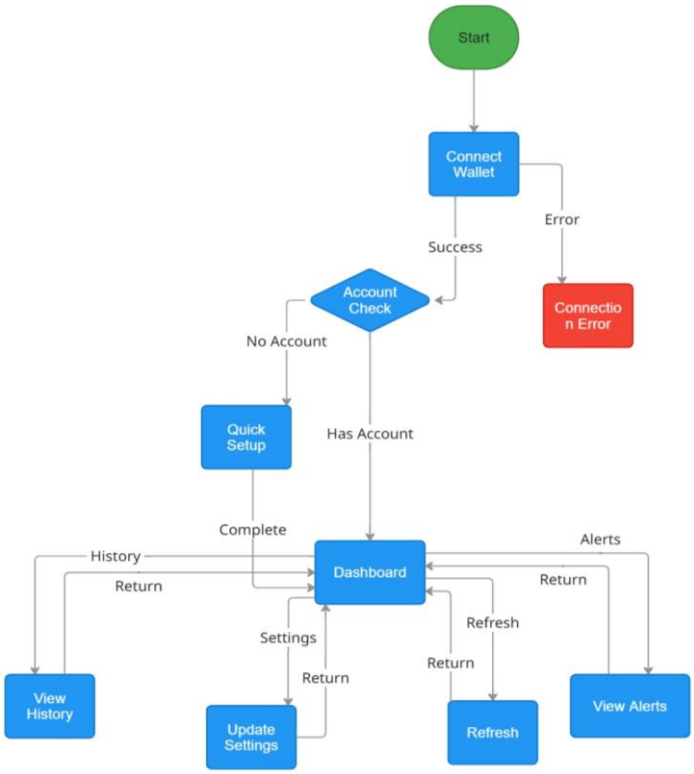



## Data Flow Architecture :




## 6) Detailed Architecture Sections :

6.1 User Interaction Flow




6.2 Risk Calculation Deep Dive :

**Risk Score Components**

**Concentration Score (40% weight)**

Herfindahl Index = Σ(token_weight$^2$)
Concentration Score = HHI / 100
Range: 0-100 (100 = single token, 0 = perfectly diversified)

**Volatility Score (40% weight)**

Weighted Volatility = Σ (token_weight × token_30d_volatility)
Normalized to 0-100 scale

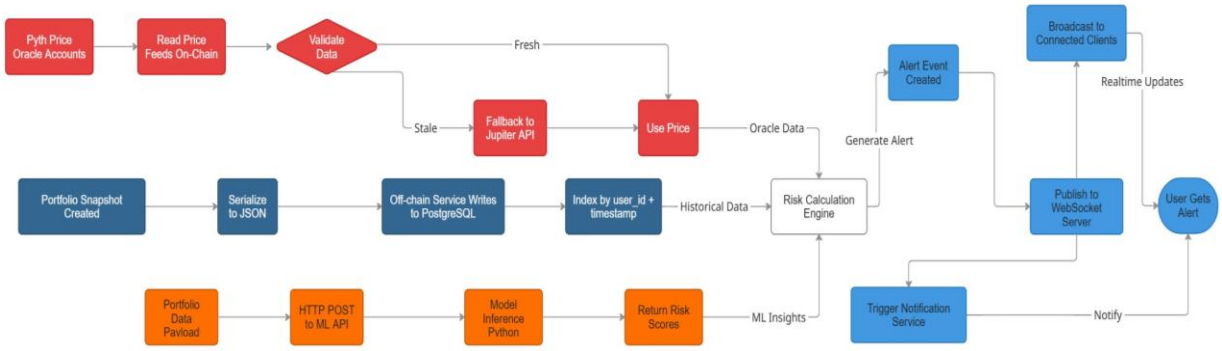
**Correlation Score (20% weight)**

Average Pairwise Correlation = Σ(correlations) / n
High correlation = High risk
Normalized to 0-100 scale

**Composite Score**

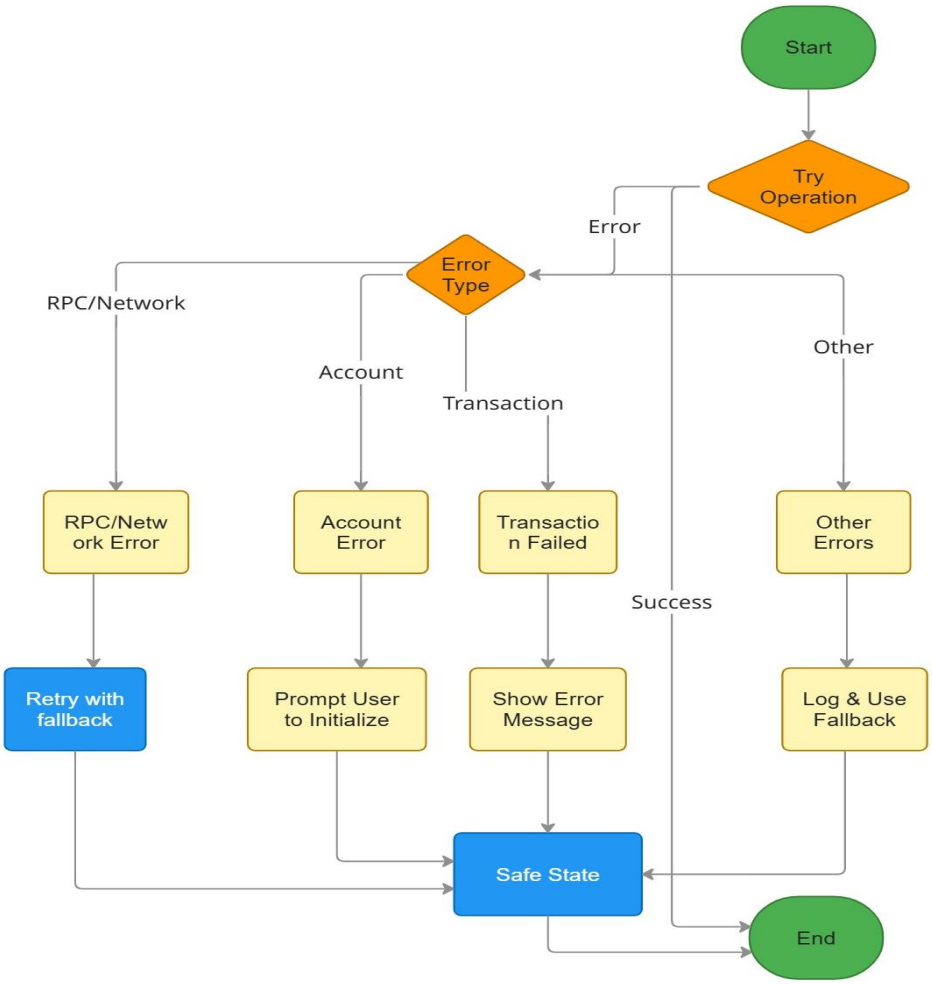
**Final Risk Score = (0.4 × Concentration) + (0.4 × Volatility) + (0.2 × Correlation)**

6.3 External Integration Detail:


Pyth Price Oracle Accounts
Read Price Feeds On-Chain
Validate Data
Fresh
Stale
Fallback to Jupiter API
Use Price
Oracle Data
Alert Event Created
Generate Alert
Broadcast to Connected Clients
Realtime Updates
Portfolio Snapshot Created
Serialize to JSON
Off-chain Service Writes to PostgreSQL
Index by user_id + timestamp
Historical Data
Risk Calculation Engine
Publish to WebSocket Server
User Gets Alert
Portfolio Data Payload
HTTP POST to ML API
Model Inference Python
Return Risk Scores
ML Insights
Trigger Notification Service
Notify

# 7. Error Handling and Edge Cases

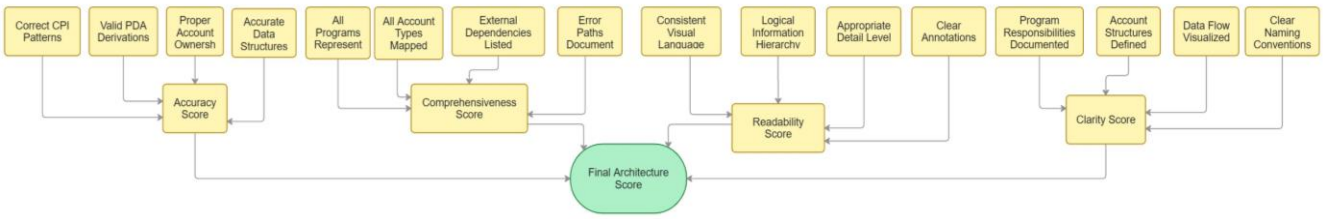
**Error Flow Diagram**




Edge Cases Handling

| Edge Case | Detection | Mitigation Strategy |
|---|---|---|
| Zero Balance Portfolio | Check `total_value_usd == 0` | Display "No assets detected" message, skip risk calculation |
| Single Token Holdings | `token_count ==1` | Set concentration score to 100, flag high risk |
| Stale Price Data | Pyth timestamp > 60s old | Use Jupiter aggregator API as fallback |
| ML Service Unavailable | HTTP timeout or 5xx error | Use statistical heuristics for risk calculation |

| | | |
|---|---|---|
| Concurrent Updates | Multiple transactions in same slot | Use versioned transactions with proper ordering |
| Alert Spam | Multiple breaches in short time | Implement 30-minute cooldown per threshold |
| Account Size Overflow | Alert history > 50 entries | Implement circular buffer, overwrite oldest |
| Network Congestion | Transaction fails repeatedly | Increase priority fee dynamically |

## 8) Evaluation Architecture :

System Evaluation Metrics